Database project: FitTrackDB

Rama Alhuthali

Shaimaa Alzahrani

Sama AlSilaimi

1. CONCEPTUAL DOCUMENTATON

1.1. Enttes

User

Represents individuals using the system.

- Atributes: user_id, frst_name, last_name, age, gender , weight, height

Food

Represents food items consumed by users.

- Atributes: food_id, name, calories

Actvity

Represents physical actvites performed by users.

- Atributes: actvity_id, name, calories_burned

1.2. Relatonships

User–Food Relatonship

- Type: Many-to-Many (M:N)
- Descripton: A user can consume multple food items, and each food item can beconsumed by multple users.
- Implementaton: user_food table

User–Actvity Relatonship

- Type: Many-to-Many (M:N)
- Descripton: A user can perform multple actvites, and each actvity can be performed by multple users.
- Implementaton: user_actvity table

Business Rules & Logic

- Identty: Users are uniquely identfed by their full name to prevent duplicate profles for the same individual.
- Data Integrity: Caloric values for both food and actvites must be positve integers.

- Automated Metrics: The system automatcally calculates a user's Basal Metabolic Rate (BMR) and recommended macronutrient distributon (Protein/Carbs/Fats) based on the Mifin-St Jeor Equaton.

2. LOGICAL DOCUMENTATON

2.1. Tables structure

Users Table

- user_id: INT, Primary Key, Auto Increment
- frst_name: VARCHAR(30), NOT NULL
- last_name: VARCHAR(30), NOT NULL
- age: INT
- gender: VARCHAR(10)
- weight: DECIMAL(5,2)
- height: DECIMAL(5,2)
- Constraint: UNIQUE(frst_name, last_name)

Food Table

- food_id: INT, Primary Key, Auto Increment
- name: 50ج), NOT NULL, UNIQUE
- calories: INT, NOT NULL, CHECK (calories > 0)

Food Table

- food_id: INT, Primary Key, Auto Increment
- name: (50), NOT NULL, UNIQUE
- calories: INT, NOT NULL, CHECK (calories > 0)

user_food Table (Juncton Table)

- user_id: INT, Foreign Key -- Users(user_id)
- food_id: INT, Foreign Key -- Food(food_id)
- Primary Key: (user_id, food_id)

user_actvity Table (Juncton Table)

- user_id: INT, Foreign Key -- Users(user_id)
- actvity_id: INT, Foreign Key -- Actvity(actvity_id)
- Primary Key: (user_id, actvity_id)

2.2 Integrity constrains

- Primary Keys: Ensure each record is uniquely identfable.
- Foreign Keys: Maintain referental integrity between related tables.
- Check Constraints:
  - calories > 0
  - calories_burned > 0
- Unique Constraints:
  - Food names must be unique.
  - Actvity names must be unique.
  - Combinaton of frst_name and last_name must be unique

3. Derived data (view)

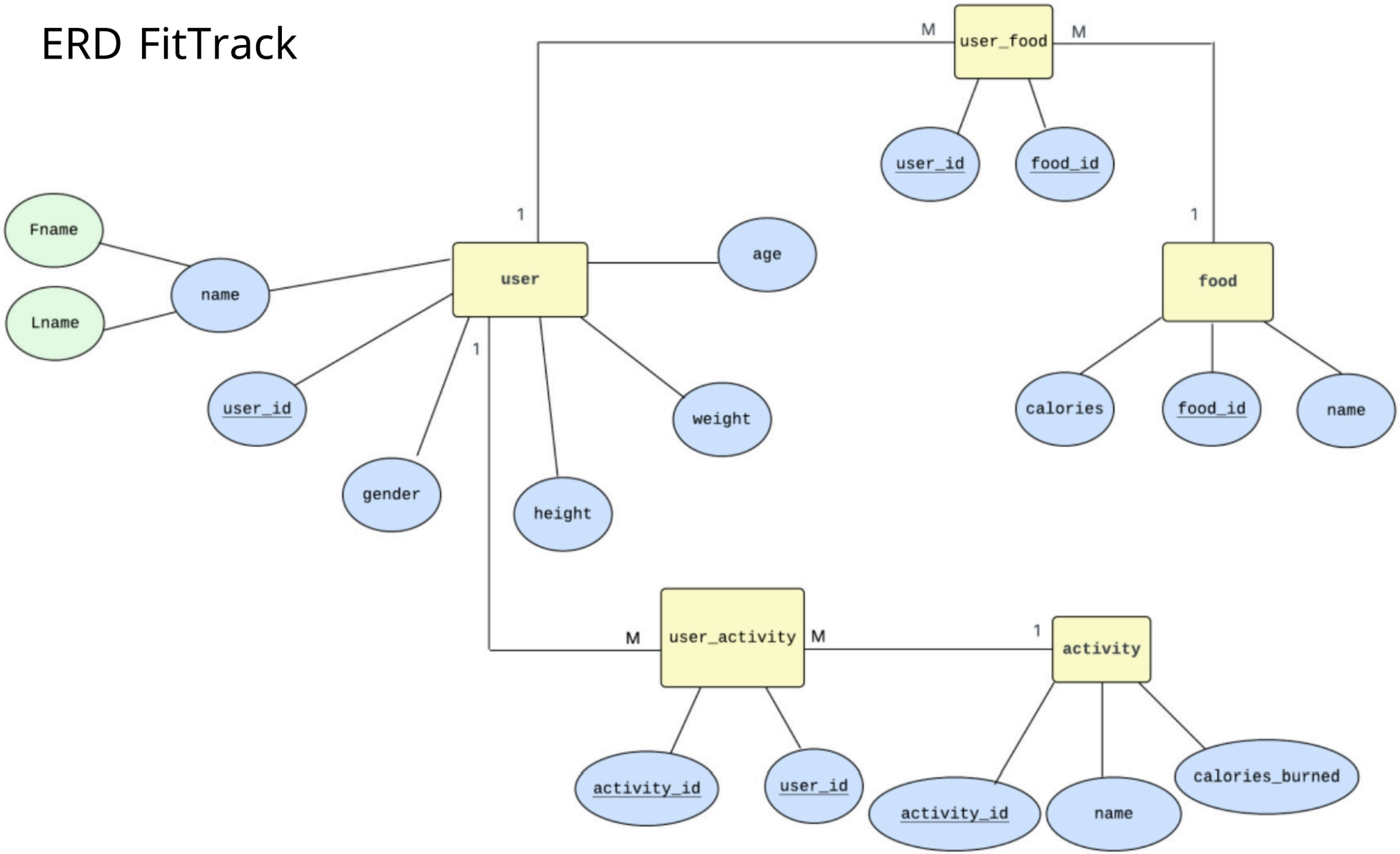
calories_report View

The calories_report view calculates daily calorie requirements and macronutrient distributon for each user based on their personal data.

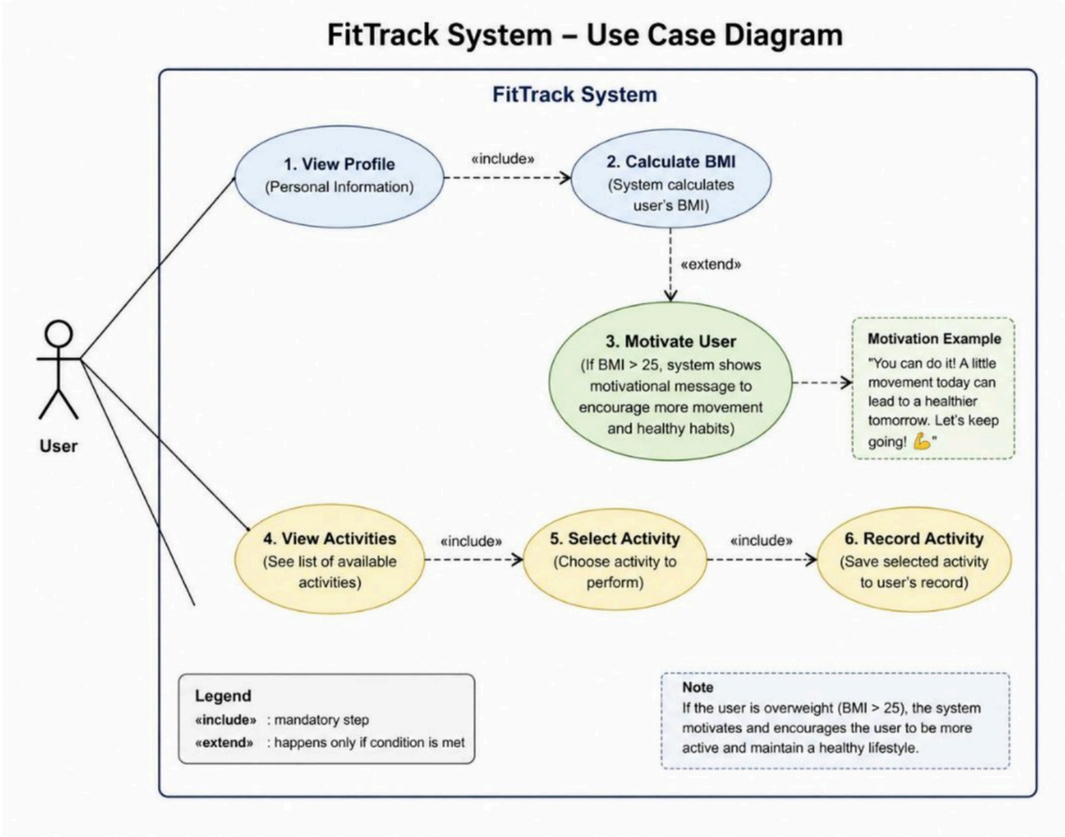
Calculaton Logic

- For males:
  - Calories = (10 × weight) + (6.25 × height) – (5 × age) + 5
- For females:
  - Calories = (10 × weight) + (6.25 × height) – (5 × age) – 161

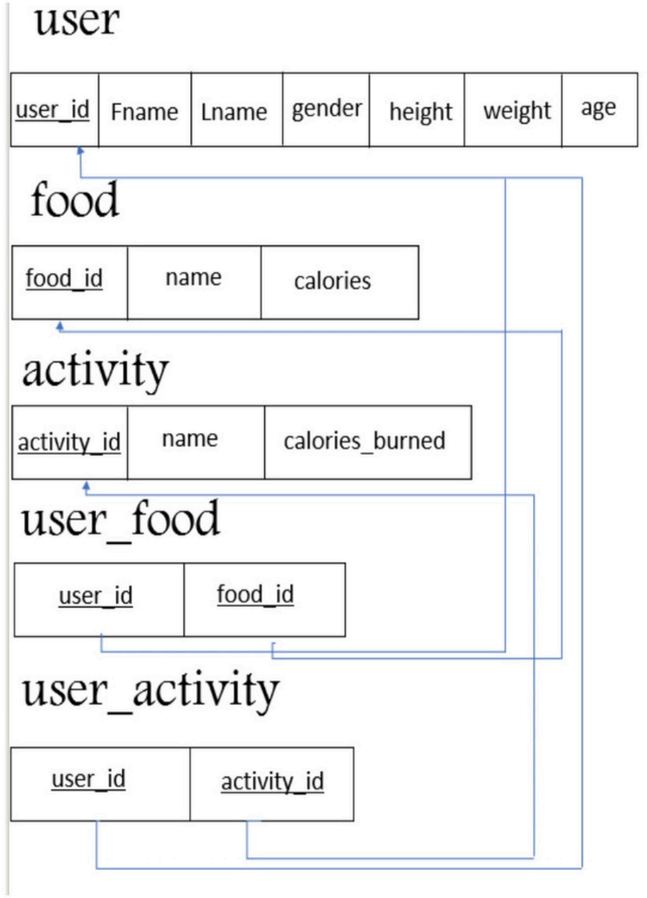
ERD FitTrack



Additional use case – motivation and recommendation system



Relation schema



Implementaton

FitTrackDB database is created and selected using USE FitTrackDB, It contains 3 tables:

1.Users: stores personal data (user_id PK, name, age, gender, weight, height) with UNIQUE
constraint on full name.

2.Food: stores food items (food_id PK, name UNIQUE, calories with CHECK > 0).

3.Actvity: stores exercises (actvity_id PK, name UNIQUE, calories_burned with CHECK > 0).
The design ensures data is organized, consistent, and valid.

```sql
CREATE DATABASE FitTrackDB;
USE FitTrackDB;
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(30) NOT NULL,
    last_name VARCHAR(30) NOT NULL,
    age INT,
    gender VARCHAR(10),
    weight DECIMAL(5,2),
    height DECIMAL(5,2),

    UNIQUE (first_name, last_name));
    CREATE TABLE Food (
    food_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    calories INT NOT NULL CHECK (calories > 0),

    UNIQUE (name)
);
CREATE TABLE Activity (
    activity_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    calories_burned INT NOT NULL CHECK (calories_burned > 0),
    UNIQUE (name)
);
```

Two juncton tables were created to represent many-to-many relatonships.

"user_food": connects users with food items.

Includes:(user_id, food_id).

Composite primary key (user_id, food_id) prevents duplicates.

Foreign keys link to Users, Food tables

user_actvity :Connects users with actvites.

Includes: (user_id, actvity_id)

Composite primary key (user_id, actvity_id) prevents duplicates.

Foreign keys link to Users and Actvity tables.

```
25    );
26    CREATE TABLE user_food(
27    user_id INT,
28    food_id INT,
29    PRIMARY KEY(user_id,food_id),
30    FOREIGN KEY(user_id)REFERENCES Users(user_id),
31    FOREIGN KEY(food_id)REFERENCES Food(food_id)
32    );
33    CREATE TABLE user_activity(
34    user_id INT,
35    activity_id INT,
36    PRIMARY KEY(user_id,activity_id),
37    FOREIGN KEY(user_id)REFERENCES Users(user_id),
38    FOREIGN KEY(activity_id)REFERENCES Activity(activity_id)
39    );
```

Data Inserton

we inserted sample records into all tables using the INSERT INTO statement.

In "Users"Table:

we added Five users with personal informaton:(frst name, last name, age, gender

,weight,height)

In " Food "Table:

we added Five food items with their calories values: (Rice, Chicken, Apple, Bread, Milk)

In "Actvity" Table:

also we added fve physical actvites:

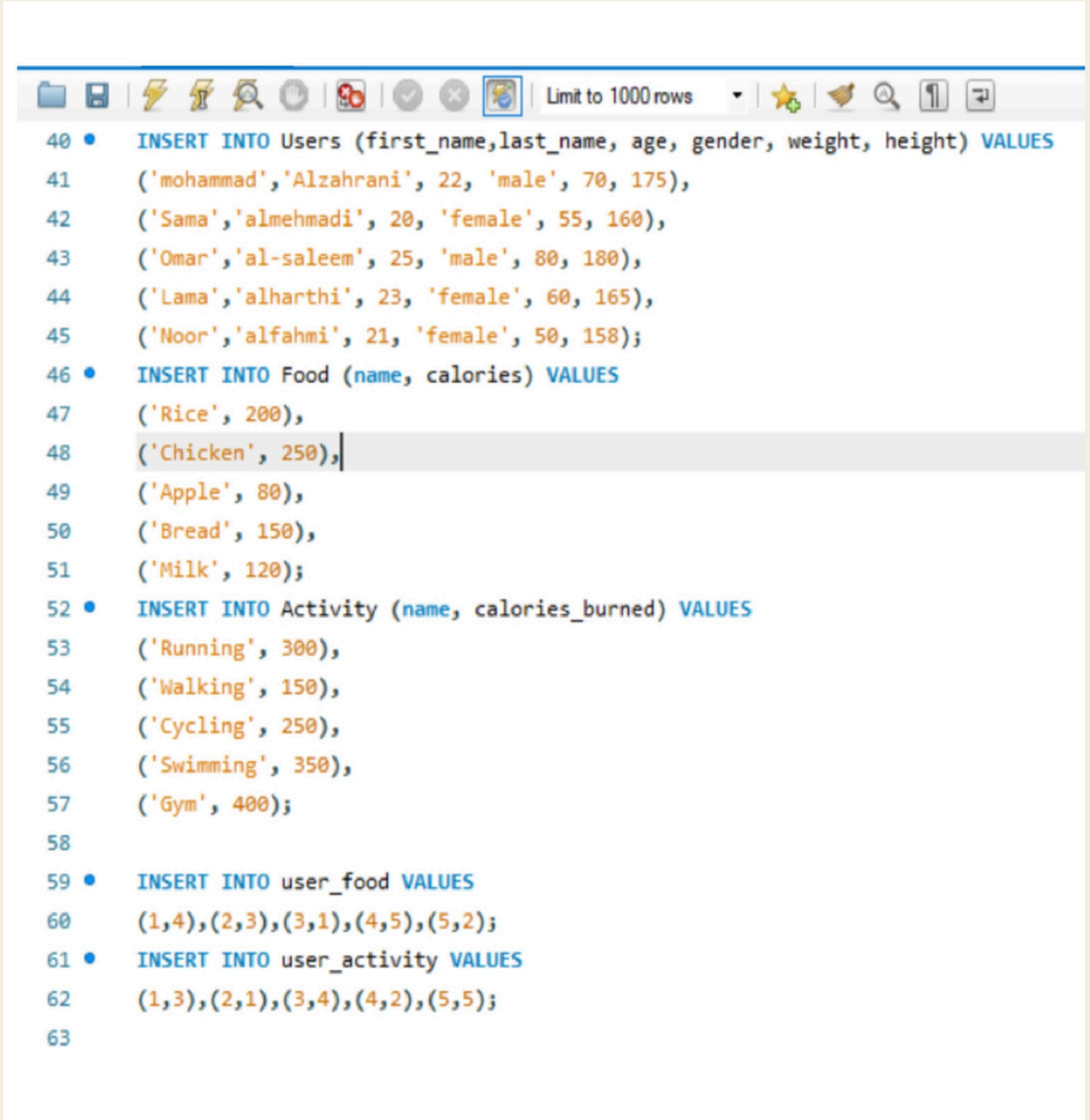
(Running, Walking, Cycling, Swimming, Gym)

In "user_food" Table:

Here we inserted records to link users with consumed food items.

In "user_actvity" Table:

we inserted records to link users with performed actvites.



Aggregate Functons

Here we use (aggregate functons) to summarize and analyze the stored data.

frst: use **"SUM(calories)"** to calculate the total calories of all food items.

Result : **800**

second: use "**(weight)**" to calculate the average weight of users.

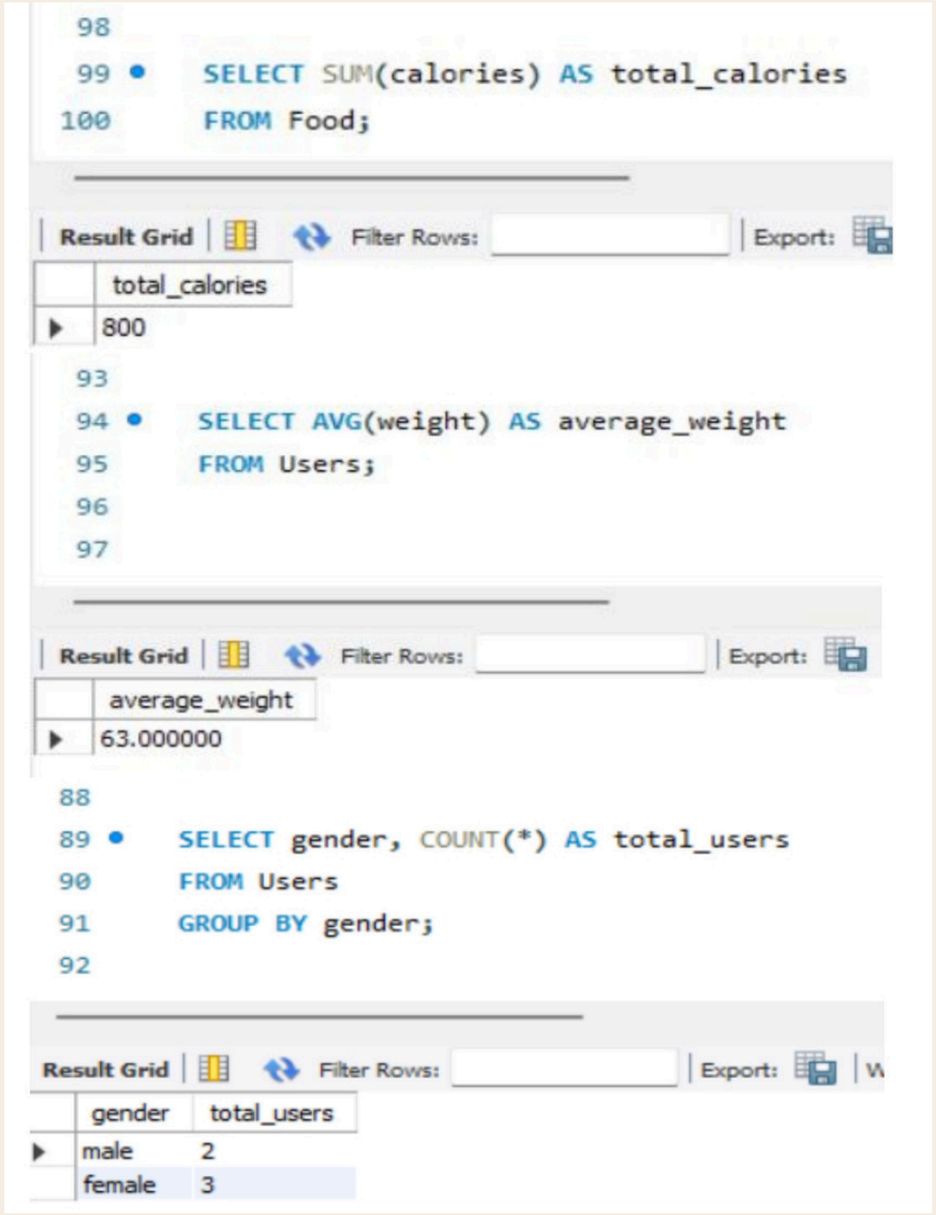
Result: **63**

third: we use "**COUNT(*)**"with "**GROUP BY gender**" to count users based on gender.

Result:

Male: 2

Female: 3



View (derived informaton)

The CASE statement is SQL’s if-then logic. It calculates the Basal Metabolic Rate (BMR) by
choosing a specifc formula based on the user's gender
It allows a single column (calories) to hold diferent calculatons for diferent types of users.



```
64  CREATE VIEW calories_report AS
65  SELECT
66      user_id,
67      first_name,
68      last_name,
69      calories,
70      (calories * 0.30) / 4 AS protein_grams,
71      (calories * 0.40) / 4 AS carbs_grams,
72      (calories * 0.30) / 9 AS fats_grams
73  FROM (
74      SELECT
75          user_id,
76          first_name,
77          last_name,
78          CASE
79              WHEN gender = 'male' THEN
80                  (10 * weight + 6.25 * height - 5 * age + 5)
81              ELSE
82                  (10 * weight + 6.25 * height - 5 * age - 161)
```

```sql
FROM (
    SELECT
        user_id,
        first_name,
        last_name,
        CASE
            WHEN gender = 'male' THEN
                (10 * weight + 6.25 * height - 5 * age + 5)
            ELSE
                (10 * weight + 6.25 * height - 5 * age - 161)
        END AS calories
    FROM Users
) AS sub;

SELECT * FROM  calories_report;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	first_name	last_name	calories	protein_grams	carbs_grams	fats_grams
1	mohammad	Alzahrani	1688.7500	126.6562500000	168.8750000000	56.2916666667
2	Sama	almehmadi	1289.0000	96.6750000000	128.9000000000	42.9666666667
3	Omar	al-saleem	1805.0000	135.3750000000	180.5000000000	60.1666666667
4	Lama	alharthi	1355.2500	101.6437500000	135.5250000000	45.1750000000
5	Noor	alfahmi	1221.5000	91.6125000000	122.1500000000	40.7166666667

This view provides a personalized calorie and nutriton report for users.

THANK YOU

Thanks For listening

Name	Works on
Rama alhuthali	(ERD, additonal use case, query implementaton, writng document)
Shaimaa Hussein Alzahrani	(query implementaton, and writng its explanaton)
Sama Eidhah AlSilaimi	(query implementaton, Relatonal